Manual técnico — PromptZal

Proyecto 2 — Laboratorio de Lenguajes Formales y de Programación

Compilador del lenguaje PromptZal implementado en Java con JFlex, parser descendente recursivo, AST, tabla de símbolos, validación semántica y generación de prompt.

1. Objetivo

PromptZal permite escribir un programa `.pz` con directivas, agentes, variables y comandos de IA. El compilador valida el programa y genera un prompt final en texto, listo para copiarse a una herramienta de IA.

El flujo completo es:

```
Archivo .pz
   ↓
Análisis léxico con JFlex
   ↓
Análisis sintáctico con parser descendente recursivo
   ↓
AST y tabla de símbolos
   ↓
Validación semántica
   ↓
Generación del prompt
   ↓
Interfaz gráfica y reportes HTML
```

2. Arquitectura general

El proyecto está organizado por fases:

```
lexico/       Tokens, lexer JFlex y lector de tokens
ast/          Nodos del árbol de sintaxis abstracta
sintactico/   Parser descendente recursivo
semantico/    Tabla de símbolos y validador semántico
generador/    Generación del prompt final
compilador/   Orquestación de todas las fases
errores/      Acumulación y clasificación de errores
reportes/     Generación de reportes HTML
ui/           Interfaz gráfica Swing
pruebas/      Programas y pruebas de consola
```

La clase `Compilador` ejecuta las fases en orden:

```
List<Token> tokens = new LectorTokens().analizar(texto, errores);
NodoPrograma programa = new Parser(tokens, errores).parsePrograma();
TablaSimbolos tabla = new ValidadorSemantico(errores).validar(programa);
String prompt = new GeneradorPromptzal().generar(programa);
return new ResultadoCompilacion(programa, tabla, errores, prompt);
```

`ResultadoCompilacion` conserva el AST, la tabla de símbolos, los errores y el prompt. La interfaz usa ese resultado para pintar las pestañas y para generar los reportes.

3. Análisis léxico con JFlex

3.1 Archivo JFlex

El archivo fuente del lexer es:

`src/main/jflex/promptzal.jflex`

JFlex genera:

`src/main/java/com/mycompany/promptzal_3/lexico/Lexer.java`

El archivo generado contiene tablas internas del autómata, mapas de caracteres y el runtime de JFlex. Por eso tiene cerca de 900 líneas y comentarios en inglés. No debe editarse a mano: si se modifica el `.jflex`, se regenera con:

```
./scripts/generar-lexer.sh
```

El script también elimina las etiquetas `<p>` que JFlex agrega en los comentarios generados.

3.2 Tabla de terminales

Terminal	Lexemas
`DIRECTIVA`	`@modelo`, `@rol`, `@formato`
`AGENTE`	`AGENTE`
`CONTEXTO`	`contexto`
`VARIABLE`	`variable`
`EJECUTAR`	`EJECUTAR`
`EXPORTAR`	`EXPORTAR`

Terminal	Lexemas
`COMANDO_IA`	`PREGUNTAR`, `GENERAR`, `RESUMIR`, `ANALIZAR`, `TRADUCIR`, `CLASIFICAR`, `EXTRAER`
`CARGAR`	`CARGAR`
`CONECTOR`	`SOBRE`, `DESDE`, `EN`, `COMO`
`FLECHA`	`->`
`IGUAL`	`=`
`MAS`	`+`
`LLAVE_A` / `LLAVE_C`	`{` / `}`
`PAR_A` / `PAR_C`	`(` / `)`
`COMA`	`,`
`ID`	Identificadores
`CADENA`	Texto entre comillas dobles
`NUMERO`	Enteros y decimales
`EOF`	Fin de archivo

Cada token guarda tipo, lexema, fila y columna. Las posiciones empiezan en 1.

3.3 Comentarios, espacios y errores

El lexer descarta:

- Espacios, tabulaciones y saltos de línea.
- Comentarios de línea `// ...`.
- Comentarios de bloque `/ ... /`.

Al llegar al final del archivo devuelve `EOF`. Si encuentra un carácter no reconocido, registra un error léxico y continúa con el siguiente token. Esto permite reportar varios errores en una sola compilación.

La clase `LectorTokens` encapsula el uso del lexer y devuelve una `List<Token>` hasta `EOF`.

4. Gramática libre de contexto

La gramática implementada es:

```
<programa>          -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas>        -> DIRECTIVA CADENA <directivas> | ε

<agentes>           -> <agente> <mas_agentes>
<mas_agentes>       -> <agente> <mas_agentes> | ε
<agente>            -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto>          -> CONTEXTO IGUAL CADENA

<variables>         -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos>          -> <comando> <mas_comandos>
<mas_comandos>      -> <comando> <mas_comandos> | ε
<comando>           -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores>        -> CONECTOR <expresion> <conectores> | ε

<expresion>         -> <termino> <mas_terminos>
<mas_terminos>      -> MAS <termino> <mas_terminos> | ε
<termino>           -> CADENA | ID | NUMERO <unidad> | <cargar>
<unidad>            -> ID | ε
<cargar>            -> CARGAR PAR_A CADENA PAR_C

<ejecuciones>       -> EJECUTAR ID <mas_ejecuciones>
<mas_ejecuciones>   -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar>          -> EXPORTAR ID <mas_ids>
<mas_ids>           -> COMA ID <mas_ids> | ε
```

La gramática está factorizada y sin recursividad por la izquierda, por lo que puede implementarse con un parser descendente recursivo.

5. Parser descendente recursivo

El parser está en:

`src/main/java/com/mycompany/promptzal_3/sintactico/Parser.java`

No usa generadores de parsers. Cada no terminal tiene un método identificable:

No terminal	Método
`<programa>`	`parsePrograma()`

No terminal	Método
`<directivas>`	`parseDirectivas()`
`<agentes>`	`parseAgentes()`
`<mas_agentes>`	`parseMasAgentes()`
`<agente>`	`parseAgente()`
`<contexto>`	`parseContexto()`
`<variables>`	`parseVariables()`
`<comandos>`	`parseComandos()`
`<mas_comandos>`	`parseMasComandos()`
`<comando>`	`parseComando()`
`<conectores>`	`parseConectores()`
`<expresion>`	`parseExpresion()`
`<mas_terminos>`	`parseMasTerminos()`
`<termino>`	`parseTermino()`
`<unidad>`	`parseUnidad()`
`<cargar>`	`parseCargar()`
`<ejecuciones>`	`parseEjecuciones()`
`<mas_ejecuciones>`	`parseMasEjecuciones()`
`<exportar>`	`parseExportar()`
`<mas_ids>`	`parseMasIds()`

El parser construye el AST mientras reconoce. Por ejemplo, `parseAgente()` devuelve un `NodoAgente` con nombre, contexto, variables, comandos y línea.

5.1 Recuperación en modo pánico

Cuando un token no coincide con lo esperado, `esperar()` reporta:

- Descripción del error.

- Fila y columna.
- Token encontrado.
- Token esperado.

Después llama a `sincronizar()`, que descarta tokens hasta encontrar uno de sincronización:

- `DIRECTIVA`
- `AGENTE`
- `LLAVE_C`
- `COMANDO_IA`
- `VARIABLE`
- `EJECUTAR`
- `EXPORTAR`
- `EOF`

También existe `sincronizarSeccion()`, que avanza hasta el inicio de otra sección del programa. El parser tiene un límite de 200 errores reportados para evitar bucles o salidas excesivas.

Gracias a esta recuperación, un archivo con varios errores sintácticos reporta todos en una sola corrida.

6. Árbol de sintaxis abstracta

Los nodos están en `ast/` y se implementan con `record` de Java:

- `NodoPrograma`: directivas, agentes, ejecuciones y exportaciones.
- `NodoDirectiva`: nombre, valor y línea.
- `NodoAgente`: nombre, contexto, variables, comandos y línea.
- `NodoContexto`: texto y línea.
- `NodoVariableDecl`: nombre, expresión y línea.
- `NodoComando`: tipo, argumento, conectores, destino y línea.
- `NodoConector`: tipo, argumento y línea.
- `NodoExpresion`: lista de términos.
- `NodoTermino`: clase, valor, unidad, carga y línea.
- `NodoUnidad`: palabra y línea.
- `NodoCargar`: archivo y línea.
- `NodoEjecucion`: agente y línea.

- `NodoExportacion`: variable y línea.

El AST conserva las líneas para que los errores y reportes puedan señalar la ubicación original.

7. Tabla de símbolos

La tabla está en `semantico/TablaSimbolos.java` y almacena objetos `Simbolo`:

```
public record Simbolo(String nombre, TipoSimbolo tipo,
        String ambito, int linea) {
}
```

Tipos de símbolo:

- `AGENTE`
- `VARIABLE_DECLARADA`
- `RESULTADO_COMANDO`

El ámbito de un agente es `global`. El ámbito de una variable o de un resultado de comando es el nombre del agente al que pertenece.

La búsqueda de variables se hace hacia atrás en la lista, lo que permite respetar el orden de declaración: una variable solo es visible después de su declaración dentro del mismo agente.

8. Validación semántica

El validador está en `semantico/ValidadorSemantico.java`.

Primero registra todos los agentes. Después recorre cada agente y:

1. Valida la expresión de cada variable antes de agregarla a la tabla.
2. Agrega la variable declarada.
3. Valida los argumentos y conectores de cada comando.
4. Agrega el destino del comando como resultado.
5. Valida las ejecuciones.
6. Valida las exportaciones.

8.1 Reglas obligatorias

- Un identificador usado en una expresión debe estar declarado antes, dentro del mismo agente.

- No puede haber dos agentes con el mismo nombre.
- `EJECUTAR` solo puede referirse a un agente existente.
- `EXPORTAR` solo puede referirse a variables que existan en algún agente ejecutado.

8.2 Excepciones

No se validan como variables:

- El primer argumento de `GENERAR` cuando es un identificador, porque indica el tipo de contenido que se va a generar.
- La palabra de unidad después de un número, por ejemplo `palabras` en `100 palabras`.
- El argumento de `EN` en `TRADUCIR ... EN <idioma>`, porque representa un idioma.

Los errores semánticos se reportan con descripción y línea. La generación del prompt puede continuar aunque existan errores.

9. Generación del prompt

El generador está en `generador/GeneradorPromptzal.java`.

El prompt incluye:

1. Encabezado con `@modelo`, `@rol` y `@formato`, si existen.
2. Por cada agente listado en `EJECUTAR`, en ese orden:

- Nombre del agente.
- Contexto.
- Variables disponibles con su origen.
- Instrucciones en lenguaje natural.

1. Lista final de resultados solicitados con `EXPORTAR`.

Solo se incluyen los agentes que aparecen en `EJECUTAR`. Los agentes declarados pero no ejecutados no se escriben en el prompt.

9.1 Traducción de comandos

Comando	Instrucción generada
`PREGUNTAR`	Basándote en el contexto o en `SOBRE`, responde la pregunta.

Comando	Instrucción generada
`GENERAR`	Genera el contenido indicado, opcionalmente desde `DESDE`.
`RESUMIR`	Resume el contenido, opcionalmente con límite `EN`.
`ANALIZAR`	Analiza el contenido considerando `SOBRE`.
`TRADUCIR`	Traduce el contenido al idioma indicado en `EN`.
`CLASIFICAR`	Clasifica el contenido en las categorías indicadas en `EN`.
`EXTRAER`	Extrae información específica desde `DESDE`.
`COMO`	Agrega el formato de entrega al final de la instrucción.

Cada comando con `-> destino` agrega una frase para guardar el resultado.

10. Reportes HTML

El generador está en `reportes/GeneradorReportes.java`.

Genera tres archivos autocontenidos en `salidas/`:

- `reporte_errores.html`: tipo, descripción, fila, columna, encontrado y esperado. Si no hay errores, lo indica explícitamente.
- `reporte_simbolos.html`: agentes, variables y resultados, con tipo, ámbito y línea.
- `reporte_ast.html`: representación del árbol con sangrías y líneas.

Los reportes se generan cuando el usuario pulsa **Abrir** en la pestaña **Reportes**. La interfaz los abre en el navegador con `Desktop.browse`.

Para la entrega se incluye un único programa de ejemplo, `entrega/ejemplo.pz`, y sus artefactos:

```
entrega/ejemplo.pz
entrega/prompt_ejemplo.md
entrega/reportes/reporte_errores.html
entrega/reportes/reporte_simbolos.html
entrega/reportes/reporte_ast.html
```

El programa contiene errores intencionales en el agente `errores`, que no se ejecuta. El agente `analista` es válido y se ejecuta, por lo que el prompt generado no se contamina con esos errores. El reporte de errores sí muestra los tres tipos: léxico, sintáctico y semántico.

11. Interfaz gráfica

La ventana principal está en `ui/VentanaPrincipal.java`.

La barra superior tiene:

- **Abrir**
- **Guardar**
- **Compilar**
- **Reportes**

El editor está en `ui/PanelEditor.java` e incluye:

- Apertura y guardado de archivos `.pz`.
- Control de cambios sin guardar.
- Números de línea.
- Resaltado de sintaxis independiente de la compilación.

La compilación se ejecuta en segundo plano con `SwingWorker` para no bloquear la interfaz.

Las pestañas de la derecha son:

- **Promptzal**: muestra el prompt y permite guardarlo como `.md` o `.txt`.
- **Errores**: tabla de errores; doble clic salta a la línea.
- **Reportes**: botones para generar y abrir los HTML.

12. Pruebas

Las pruebas de consola están en `src/main/java/.../pruebas/`:

- `PruebaLexico`
- `PruebaParser`
- `PruebaSemantica`
- `PruebaGenerador`
- `PruebaReportes`

Archivos de prueba:

- `referencia.pz`: programa válido con todos los comandos.
- `lexico_errores.pz`: errores léxicos.
- `sintactico_errores.pz`: errores sintácticos.

- `semanticos.pz`: los cuatro errores semánticos obligatorios.
- `ambitos.pz`: variable fuera de su agente.
- `declaracion_tardia.pz`: uso antes de declarar.
- `excepciones.pz`: excepciones de `GENERAR`, unidades y `TRADUCIR`.
- `exportar_inexistente.pz`: exportación de una variable inexistente.

Comandos:

```
mvn clean package
java -cp target/classes com.mycompany.promptzal_3.pruebas.PruebaLexico pruebas/referencia.pz
java -cp target/classes com.mycompany.promptzal_3.pruebas.PruebaParser pruebas/referencia.pz
java -cp target/classes com.mycompany.promptzal_3.pruebas.PruebaSemantica pruebas/semanticos.pz
java -cp target/classes com.mycompany.promptzal_3.pruebas.PruebaGenerador pruebas/referencia.pz
pruebas/prompt_esperado.md
java -cp target/classes com.mycompany.promptzal_3.pruebas.PruebaReportes pruebas/referencia.pz
```

13. Compilación y ejecución

```
mvn clean package
java -jar target/PromptZal_3-1.0-SNAPSHOT.jar
```

El proyecto usa Java 21, FlatLaf para el tema visual y Maven Shade para generar un JAR ejecutable.

14. Conclusión

El compilador cubre las fases solicitadas: léxico con JFlex, sintáctico con parser descendente recursivo, AST, tabla de símbolos, validación semántica, generación del prompt y reportes HTML. La interfaz integra todas las fases y permite revisar el resultado y los errores de una compilación.